

AUTOMATED PRODUCE SCANNER

Software Setup and System Architecture

Real-Time Pricing and Packaging System

Portfolio / Technical Documentation Edition

Acknowledgement

The successful completion of this project would not have been possible without the collective effort, knowledge, cooperation, and support of the individuals who contributed throughout its development.

First and foremost, I would like to express my sincere gratitude to my colleagues and fellow engineers, **Maria Jeni Louise Brillas, Jean Rose Jamellano, Jeston Roem Lacson, and Kane Maxine Aesha M. Magan**, for their cooperation, dedication, ideas, and valuable contributions to the fulfillment of this project. The knowledge and skills that we shared throughout the development process played an important role in overcoming challenges and bringing the project to completion.

I would also like to extend my heartfelt appreciation to my **family, mentors, friends, and other individuals** who provided encouragement, guidance, assistance, and support throughout our academic journey and the development of this project. Their continued support served as a source of motivation, especially during challenging stages of the project.

My sincere gratitude is also extended to the **faculty and staff of Carlos Hilado Memorial State University**, particularly those who provided us with the knowledge, guidance, resources, and opportunities necessary to develop and demonstrate the skills we acquired throughout our years of study. Their instruction and mentorship contributed greatly to our growth as aspiring engineers.

Above all, I give my deepest gratitude to **Almighty God** for His guidance, strength, wisdom, and protection throughout the entire process. His blessings gave us the perseverance and determination to overcome the challenges we encountered and successfully complete this project.

To everyone who, in one way or another, contributed to the completion of this work, **thank you sincerely**. This achievement would not have been possible without your support and contributions.

May God bless us all.

1. Software Setup Overview

The software architecture of the Automated Produce Scanner is centered on a Raspberry Pi 4 Model B running the image-detection, weighing, pricing, display, transaction, and packaging control logic. The project uses Python for system implementation, OpenCV for image handling, system-level camera commands for the modern libcamera framework, and an Edge Impulse model for offline image inference.

The software is designed to connect computer vision with weight sensing and a locally stored pricing database. After a valid produce detection, the system combines the detected vegetable type, measured weight, and stored price-per-kilogram value to calculate and display the transaction price in real time.

2. Software Architecture



Figure 1. Functional software architecture.

The architecture is a portfolio-level representation of the software flow described in the capstone documentation. It is not intended to replace the original source code or a detailed UML/system diagram.

3. Software Environment

Software / Environment	Role in the System
Operating System	Debian 13 (Trixie) on Raspberry Pi, as specified in the implementation documentation.
Programming Language	Python.
Computer Vision	OpenCV for image handling and preprocessing.
Camera Framework	Modern libcamera framework, controlled through system-level camera commands.
Machine Learning Platform	Edge Impulse Studio for image dataset processing, model training, evaluation, and deployment.
Inference Format	Linux (AArch64) Edge Impulse deployment package (.eim) with INT8 quantization for offline Raspberry Pi inference.
ML Model	Transfer-learning-based image detection model; the system architecture identifies a FOMO-based object-detection model.
Database	Local database storing vegetable pricing and transaction-related information.
User Interface	LCD output for produce type, confidence/accuracy, weight, and price; administrator interface for price updates.
Transaction Output	Thermal barcode printer for printed transaction labels/barcodes.
System Control	Python logic and system-level commands coordinating detection, weighing, pricing, display, printing, and packaging.

4. Dataset and Machine-Learning Setup

The image dataset contains the produce classes red onion, garlic, and marble potato, together with an empty class representing the absence of produce. The system also accounts for an assorted-produce condition when two or more different vegetable classes exceed the configured confidence threshold in the same frame.

- Capture image data using the Raspberry Pi Camera Module V2 mounted at a fixed height above the platform.
- Use the blue background platform and stable daylight arrangement documented for image collection.
- Upload and label images in Edge Impulse Studio.
- Train and evaluate the transfer-learning-based image detection model.
- Export the trained model as a Linux (AArch64) Edge Impulse deployment package (.eim).

- Deploy the model to the Raspberry Pi for offline inference using INT8 quantization.

5. Image-Processing and Inference Pipeline

Image acquisition: The camera captures the produce image using the Raspberry Pi camera subsystem and modern libcamera framework.

Image handling: Python and OpenCV are used to handle the image data.

Preprocessing: The documented processing pipeline resizes the image to 160×160 pixels and applies normalization.

Inference: The processed image is passed to the Edge Impulse executable/model. The documented architecture uses a FOMO-based object-detection model.

Confidence evaluation: The inference engine returns a predicted class label and confidence score. A predefined confidence threshold determines whether a detection is valid.

Single-item validation: A single vegetable class exceeding the threshold is accepted as a valid produce detection.

Assorted-item validation: If two or more different vegetable classes exceed the threshold in the same frame, the software identifies an “assorted vegetable” condition and prevents further processing.

No-produce handling: If the expected produce condition is not detected, the system remains in its waiting/standby logic rather than proceeding with pricing.

6. Weight-Sensing Software Flow

The weighing subsystem receives digital weight data from the load cell through the HX711 module. The software uses the measured weight as an input to real-time price calculation. A tare button is provided so the weighing system can be zeroed before measurement.

- Initialize the weighing subsystem.
- Apply tare/zeroing when requested.
- Read weight data from the load-cell/HX711 subsystem.
- Use the valid produce classification together with the measured weight.
- Continue monitoring for changes when the user changes the quantity of produce.
- Return to image processing and re-weigh/recalculate when the input changes.

7. Real-Time Pricing Software

Once a valid vegetable class and weight are available, the software retrieves the corresponding price per kilogram from the local database. The measured weight and stored unit price are combined to calculate the total price, which is then shown on the LCD.

$$\text{Total Price} = \text{Measured Weight} \times \text{Stored Price per Kilogram}$$

The capstone documentation reports that the price computation module achieved 100% accuracy in the reported trials when values were rounded to two decimal places.

8. Local Database and Administrator Interface

The system uses a local database to store vegetable pricing and transaction-related information. An administrator interface allows authorized personnel to update vegetable prices. Updated prices are stored in the local database and are reflected in subsequent real-time price calculations without requiring image-model retraining.

- Maintain a record for each supported vegetable and its assigned unit price per kilogram.
- Allow authorized personnel to update pricing values.
- Use the current stored price during customer transactions.
- Keep pricing functionality independent from the image-classification model so price changes do not require retraining.

9. User Interface and Output Software

Software Output / Interface	Function
LCD	Displays detected vegetable type, confidence/accuracy, measured weight, and calculated price.
Thermal Barcode Printer	Receives transaction information and prints a barcode label containing item details, weight, and price.
User Confirmation	The OK button finalizes the transaction after the user confirms the displayed information.
Error Handling	The interface responds to conditions such as no produce detected and multiple vegetables detected.
Administrative Interface	Allows authorized personnel to update vegetable prices stored in the local database.

10. Barcode Generation and Transaction Processing

After the transaction is confirmed, the software initiates barcode printing. The generated barcode contains transaction information used for later scanning and database verification. The reported tests showed that generated barcodes were printed clearly, scanned successfully, and matched their corresponding database records.

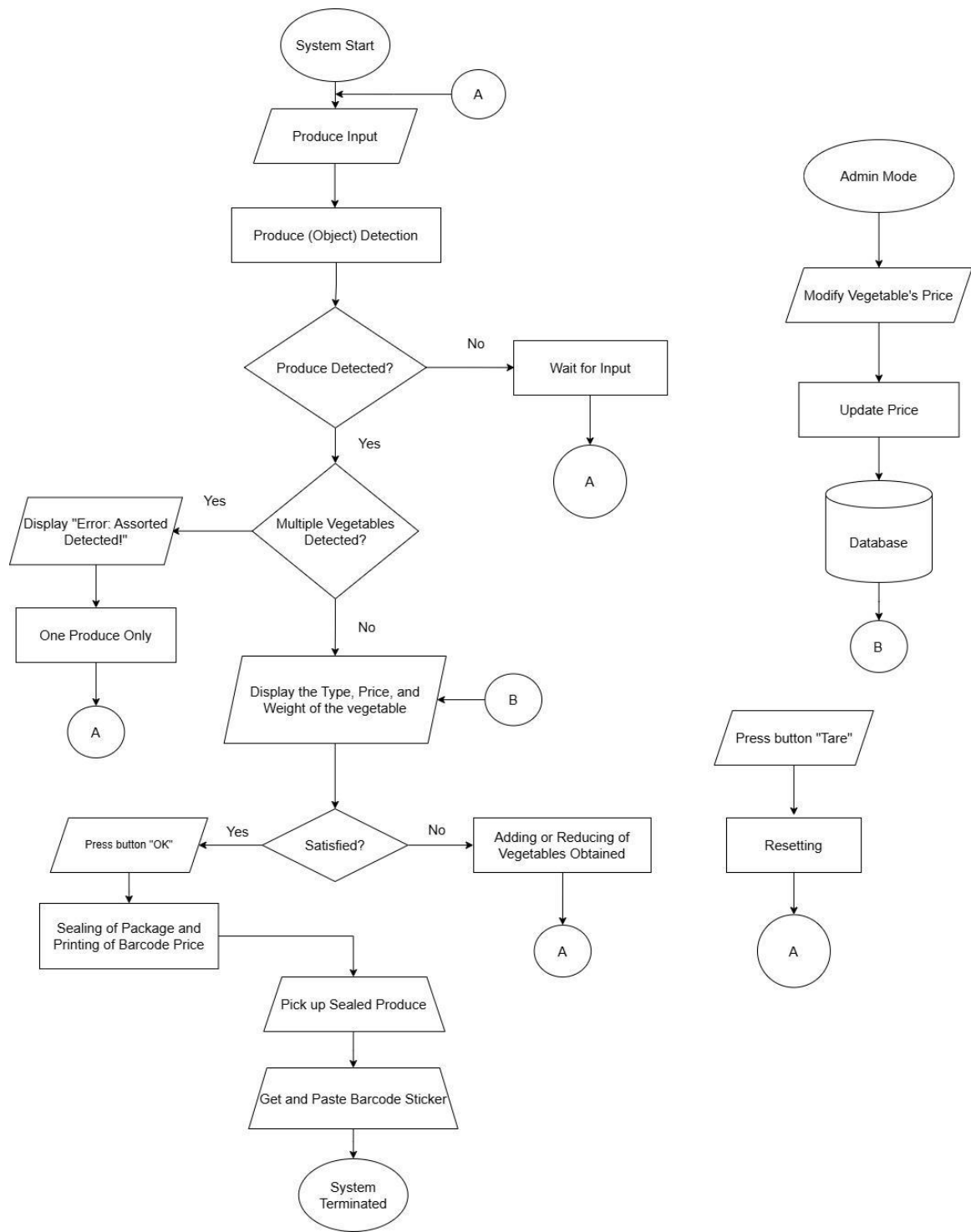
1. Finalize the item, weight, and computed price after user confirmation.
2. Generate a barcode representing the transaction/item information.
3. Send the barcode and transaction details to the thermal printer.
4. Print the barcode label.
5. Allow the printed barcode to be scanned for transaction verification or cashier-side processing.

11. Packaging Control Software

The Raspberry Pi controls the packaging sequence after the user confirms the transaction. The software activates the sealing mechanism, which uses the linear actuator to operate the impulse sealer. The sealed product is then released to the user.

6. Wait for a valid produce detection and weight.
7. Calculate and display the transaction.
8. Wait for user confirmation.
9. Activate the sealing mechanism.
10. Operate the actuator and impulse sealer.
11. Initiate barcode printing.
12. Complete the transaction and return the system to the next transaction state.

12. Complete Software Workflow



13. Software Setup / Installation Sequence

The thesis establishes the software environment and deployment approach but does not provide a complete command-by-command installation guide. The following sequence therefore describes the documented setup workflow without inventing package names, GPIO libraries, database engines, or exact shell commands that are not specified in the source.

Prepare Raspberry Pi OS environment: Use the documented Debian 13 (Trixie) environment on the Raspberry Pi.

Prepare Python environment: Set up the Python runtime used by the system implementation.

Prepare camera software: Use the modern libcamera framework and the system-level camera control approach documented in the thesis.

Install/prepare OpenCV: Prepare OpenCV for image handling and preprocessing.

Prepare Edge Impulse deployment: Export the trained Linux (AArch64) .cim model from Edge Impulse Studio and deploy it to the Raspberry Pi.

Prepare local database: Create/initialize the local price database and populate the supported vegetable prices.

Configure LCD output: Prepare the software interface that displays classification, weight, and price.

Configure printer output: Prepare the barcode/thermal-printer integration.

Configure packaging control: Prepare the software logic that activates the actuator/sealing mechanism after confirmation.

Run module tests: Verify each software/hardware subsystem before full integration.

Run integrated tests: Verify image detection, weight sensing, price computation, display, barcode printing, and packaging as one workflow.

14. Software Testing and Validation

Software Test	Validation Focus
Classification / Detection	Evaluate predicted vegetable labels and confidence scores using repeated image trials.
Assorted Detection	Verify that multiple different produce classes trigger the assorted-vegetable error condition.
Weight Processing	Compare software weight readings with reference measurements and monitor measurement error.
Price Computation	Compare calculated prices with expected values after

	applying the stored price per kilogram.
LCD Output	Verify that produce type, accuracy, weight, and price are displayed correctly.
Barcode Printing	Verify generation, print clarity, scanning, and database matching.
Error Handling	Verify no-produce and multiple-produce conditions.
System Stability	Run repeated integrated trials and observe response time and stability.

15. Reported Software Performance

- The documented system achieved approximately 80% overall vegetable classification accuracy, with performance varying by vegetable type.
- Assorted vegetable detection was reported at 100% accuracy.
- Price computation was reported at 100% accuracy in the documented trials when results were rounded to two decimal places.
- Barcode generation, printing, scanning, and database matching were reported as 100% accurate in the documented trials.
- The system operated offline for Edge Impulse inference using the deployed AArch64 .eim package.
- Weight readings generally remained within the documented acceptable tolerance, although minor fluctuations were observed.

16. Software Limitations

- The image model can identify only the trained produce classes: red onion, garlic, and marble potato, plus the empty/no-produce class.
- The image detector does not determine whether a vegetable is overripe or damaged.
- Classification performance varies between produce types, with visually similar classes requiring further dataset/model improvement.
- The system operates as a standalone unit with a local database and does not support multi-site or centralized integration.
- The system is designed for sequential operation.
- The thesis does not provide a complete command-by-command installation procedure or a complete source-code listing in the relevant methodology sections.

17. Recommended Future Software Improvements

- Expand the image dataset, particularly for visually similar vegetables such as marble potato and garlic.

- Include greater variation in lighting, object positioning, and backgrounds during training.
- Use controlled lighting or an enclosed imaging environment to reduce environmental variation.
- Apply weight filtering/noise-reduction techniques to reduce fluctuations.
- Continue improving the sealing-control software together with the mechanical alignment and heat regulation.
- Consider future expansion of the local database/system architecture if centralized or multi-site operation is required.

18. Documentation Note

This software setup document is a portfolio-oriented technical adaptation of the submitted capstone thesis. It preserves the software technologies and architecture explicitly described by the source. Where the thesis does not specify exact installation commands, Python package versions, database engine names, GPIO mappings, or complete source-code structure, those details have deliberately not been invented.